



Sistema de Gestión de Pruebas Integrales para Aplicación Web:

Proyecto MAP Parking

Sergio Andres Ordoñez Diaz

Hermes Pascuas Herrera

Corporación Universitaria del Huila – CORHUILA

Programa de Ingeniería de Sistemas

Calidad de Software – 2026A

José Miguel Llanos Mosquera

Mayo de 2026



Tabla de Contenido

Contenido

Tabla de Contenido	2
Índice de Tablas	4
Índice de Figuras	5
Resumen	6
1. Introducción	7
2. Objetivos.....	9
2.1 Objetivo General	9
2.2 Objetivos Específicos	9
3. Marco Teórico y Estándares Aplicados.....	11
3.1 ISO/IEC 25010 – Modelo de Calidad del Producto Software	11
3.2 ISO/IEC 29119 – Estándar Internacional para Pruebas de Software	11
3.3 ISO 9001 e Ingeniería de Calidad	12
REQUISITOS FUNCIONALES.....	12
RF-01. Gestión de roles de usuario (HU_01)	13
RF-02. Registro de nuevos usuarios (HU_01 / TC-03).....	13
RF-03. Autenticación de usuarios (HU_02 / TC-01, TC-02)	13
RF-04. Gestión de sesión activa (HU_02 / TC-07).....	13
RF-05. Consulta de servicios y tarifas (HU_02 / TC-04, TC-05).....	13
RF-06. Gestión de reservas de parqueadero (HU_03 y HU_05 / TC-06)	13
RF-07. Registro de ingreso y salida de vehículos (HU_04)	13
RF-08. Generación automática de tarifas (descripción del sistema, sec. 4.1)	14
RF-09. Pago del servicio de parqueadero (HU_06)	14
RF-10. Exposición de API REST (descripción técnica)	14
3.4 CMMI – Modelo de Madurez de Capacidades	14
4. Desarrollo: Plan y Especificación de Pruebas	16
4.1 Descripción del Sistema MAP Parking	16
4.2 Gestión del Proyecto con Tablero Kanban	17
4.3 Arquitectura y Entornos de Prueba	18
4.3.1 Entorno de Producción / Desarrollo Local	19
4.3.2 Entorno de Pruebas y CI/CD (H2 In-Memory).....	19



4.4 Plan de Pruebas (ISO/IEC 29119-3).....	20
4.5 Especificación de Casos de Prueba	22
4.5.1 Casos de Prueba Funcionales — Selenium WebDriver (POM).....	22
4.5.2 Escenarios de Prueba de Rendimiento — Apache JMeter.....	23
5. Resultados Obtenidos y Reportes de Métricas	24
5.1 Reporte de Ejecución — Pruebas Unitarias (JaCoCo)	24
5.2 Análisis Estático de Calidad — SonarQube	25
5.2.1 Hallazgos Iniciales del Análisis.....	26
5.2.2 Resultados Finales tras Refactorización	26
5.3 Reporte de Ejecución — Pruebas de Rendimiento (JMeter)	28
5.4 Reporte de Ejecución — Pruebas Funcionales (Selenium WebDriver)	31
5.5 Integración Continua y Entrega Continua (GitHub Actions)	32
5.6 Métricas de Defectos y Densidad de Defectos.....	33
6. Relación con ISO/IEC 25010	36
7. Conclusiones y Lecciones Aprendidas.....	38
7.1 Aislamiento de Entornos como Estrategia Fundamental de CI/CD	38
7.2 Robustez Arquitectural Demostrada por Métricas Objetivas	38
7.3 La Automatización Transforma la Calidad en una Cultura Organizacional.....	39
7.4 SonarQube como Motor de Mejora Continua del Código	39
7.5 Trazabilidad como Principio de Calidad Verificable	39
Referencias	41



Índice de Tablas

Tabla 1 Stack Tecnológico de MAP Parking	16
Tabla 2 Estado de Historias de Usuario — Tablero Trello MAP Parking G2	17
Tabla 3 Niveles de Prueba Implementados en MAP Parking	20
Tabla 4 Plan de Pruebas Completo — Criterios de Entrada, Ejecución y Salida (ISO/IEC 29119-3)	21
Tabla 5 Casos de Prueba Funcionales — Selenium WebDriver con Trazabilidad HU	22
Tabla 6 Especificación de Escenarios de Prueba de Rendimiento — Apache JMeter	24
Tabla 7 Resultados Finales del Análisis SonarQube — Overall Code	27
Tabla 8 Métricas de Rendimiento Apache JMeter — Dashboard de Resultados	29
Tabla 9 Densidad de Defectos y Métricas de Calidad del Código — MAP Parking	34
Tabla 10 Mapeo ISO/IEC 25010 vs. Métricas Obtenidas — MAP Parking	36



Índice de Figuras

Figura 1 Tablero Kanban MAP Parking G2 en Trello.....	18
Figura 2 Comparativa de configuraciones de conexión a base de datos para los entornos de producción y pruebas	20
Figura 3 Reporte de cobertura de código JaCoCo — 83%	25
Figura 4 Hallazgos iniciales en SonarQube antes de la refactorización del código	26
Figura 5 Resultados finales del análisis estático en SonarQube tras la refactorización del código.	28
Figura 6 Resultados del dashboard de Apache JMeter	31
Figura 7 Reporte Surefire de Selenium WebDriver — 7 pruebas ejecutadas sin errores.	32
Figura 8 Pipeline de Github Actions, todos los jobs exitosos	33

Resumen

El presente trabajo documenta la implementación de un Sistema Integral de Gestión de Pruebas de Software para la aplicación web MAP Parking, desarrollada con Java 17 y Spring Boot en el backend, Angular en el frontend y MySQL como motor de base de datos relacional en el entorno de producción. El proyecto aplica los estándares internacionales ISO/IEC 25010, ISO/IEC 29119, ISO 9001 y el modelo CMMI, integrando herramientas modernas de QA/QC que cubren cuatro niveles de validación: pruebas unitarias con JUnit 5 y JaCoCo (cobertura del 84.2%); análisis estático con SonarQube (calificación A en Security, Reliability y Maintainability); pruebas funcionales E2E con Selenium WebDriver bajo el patrón Page Object Model (7 casos ejecutados con 0 fallos); y pruebas de rendimiento con Apache JMeter (975 peticiones, 0.00% de tasa de error, Apdex 1.000). La infraestructura fue orquestada con Docker y un pipeline de CI/CD en GitHub Actions que automatiza toda la validación con cada commit. Los resultados evidencian el cumplimiento de las características de calidad definidas por la ISO/IEC 25010: confiabilidad, eficiencia de desempeño, mantenibilidad, seguridad, adecuación funcional y portabilidad.

Palabras clave: calidad de software, pruebas de software, JaCoCo, SonarQube, Selenium WebDriver, Apache JMeter, GitHub Actions, Docker, ISO/IEC 25010, ISO/IEC 29119, cobertura de código, defect density.

1. Introducción

El aseguramiento de calidad del software (QA) y el control de calidad (QC) representan dos de los pilares más importantes del ciclo de vida del desarrollo de sistemas modernos. En un mundo donde las aplicaciones informáticas sustentan procesos críticos de negocio, salud, finanzas y transporte, la ausencia de un plan formal de pruebas puede derivar en fallos catastróficos con impactos económicos, legales y reputacionales de gran magnitud. La norma ISO/IEC 25010 establece que la calidad de un producto software no es un atributo único, sino un conjunto multidimensional de características que incluyen confiabilidad, eficiencia de desempeño, seguridad, mantenibilidad, portabilidad y adecuación funcional.

El presente informe documenta la implementación completa de un Sistema Integral de Gestión de Pruebas para MAP Parking, una aplicación web diseñada para la gestión de un parqueadero. El sistema permite a los usuarios registrarse, autenticarse según roles diferenciados, gestionar reservas de espacios de estacionamiento, registrar el ingreso de vehículos y realizar el pago del servicio. Técnicamente, MAP Parking está construido con Java 17 y Spring Boot en el backend (API REST con arquitectura MVC), Angular en el frontend (SPA responsiva), y MySQL como motor de base de datos en producción. Para el entorno de pruebas automatizadas, se utiliza una base de datos H2 en memoria que permite ejecutar el pipeline de CI/CD sin dependencias externas. La estrategia de calidad implementada se articula en cuatro frentes complementarios e integrados. En primer lugar, se diseñaron e implementaron pruebas unitarias con JUnit 5 y Mockito, instrumentadas mediante JaCoCo para medir y reportar la cobertura de código, superando el umbral mínimo del 80% con un resultado de 84.2%. En segundo lugar, se realizó un análisis estático del código fuente con SonarQube, identificando y corrigiendo Code Smells,



vulnerabilidades de seguridad y problemas de mantenibilidad, alcanzando calificación A en todas las categorías tras la refactorización. En tercer lugar, se automatizaron pruebas funcionales E2E con Selenium WebDriver 4.x bajo el patrón Page Object Model, ejecutando 7 casos de prueba que cubrieron los flujos críticos del sistema sin ningún fallo. Finalmente, se ejecutaron pruebas de rendimiento, carga y estrés con Apache JMeter en modo consola, procesando 975 peticiones HTTP con una tasa de error del 0.00% y un índice Apdex perfecto de 1.000.

Todo el ecosistema de pruebas fue orquestado mediante un pipeline de Integración Continua y Entrega Continua (CI/CD) implementado en GitHub Actions, utilizando contenedores Docker para garantizar la reproducibilidad del entorno en cualquier plataforma. El proyecto se enmarca metodológicamente en los estándares ISO/IEC 25010 (calidad del producto), ISO/IEC 29119 (procesos de prueba), ISO 9001 (gestión de la calidad) y el modelo CMMI (madurez de capacidades). La gestión del trabajo se realizó mediante un tablero Kanban en Trello, que permitió hacer seguimiento a las historias de usuario desde su definición hasta su validación.



2. Objetivos

2.1 Objetivo General

Aplicar de forma integral los conceptos, estándares internacionales (ISO/IEC 25010, ISO/IEC 29119, ISO 9001, CMMI) y herramientas modernas de QA/QC en el desarrollo y validación de una aplicación web funcional en Java, implementando un Sistema de Gestión de Pruebas que garantice la calidad del software de forma automatizada, medible y reproducible.

2.2 Objetivos Específicos

- Diseñar e implementar un Plan de Pruebas formal siguiendo los lineamientos de la norma ISO/IEC 29119-3, que contemple pruebas unitarias, funcionales, de rendimiento y análisis estático.
- Implementar pruebas unitarias con JUnit 5 y Mockito, alcanzando una cobertura mínima del 80% en los paquetes de lógica de negocio, reportada mediante JaCoCo con evidencia objetiva y verificable.
- Desarrollar pruebas funcionales de interfaz de usuario con Selenium WebDriver 4.x bajo el patrón Page Object Model (POM), cubriendo los flujos críticos de la aplicación con trazabilidad hacia las historias de usuario.
- Ejecutar pruebas de rendimiento, carga y estrés con Apache JMeter en modo consola, evaluando el comportamiento del backend bajo tres perfiles de concurrencia progresiva y documentando los resultados con métricas objetivas.



- Analizar la calidad del código fuente con SonarQube, reportando métricas de mantenibilidad, seguridad, confiabilidad, cobertura y densidad de defectos, y aplicar las correcciones necesarias.
- Implementar un pipeline de CI/CD en GitHub Actions con Docker que automatice la ejecución de la suite completa de pruebas ante cada push al repositorio, garantizando la validación continua del software.
- Establecer trazabilidad bidireccional entre los requerimientos funcionales (HU_01 a HU_06), los casos de prueba y los resultados de ejecución, cumpliendo el principio de trazabilidad de ISO/IEC 29119.
- Calcular y analizar métricas de calidad incluyendo cobertura de código, densidad de defectos, tasa de defectos aceptados y métricas de rendimiento, relacionándolas con el modelo de calidad ISO/IEC 25010.

3. Marco Teórico y Estándares Aplicados

3.1 ISO/IEC 25010 – Modelo de Calidad del Producto Software

La norma ISO/IEC 25010 (2011) define el modelo de calidad del producto software como un conjunto estructurado de ocho características que determinan qué tan bien un sistema satisface las necesidades de sus usuarios y partes interesadas. Cada característica se subdivide en subcaracterísticas medibles que permiten evaluar la calidad de forma objetiva y cuantificable.

La Confiabilidad establece que el sistema debe ejecutar las funciones especificadas bajo condiciones determinadas durante un período de tiempo definido, sin fallos ni degradaciones. La Eficiencia de Desempeño evalúa el rendimiento en relación con los recursos utilizados, incluyendo el tiempo de respuesta, la capacidad y el comportamiento bajo carga. La Mantenibilidad determina la facilidad con la que el software puede ser modificado, corregido o mejorado, considerando factores como la modularidad, la reusabilidad y la testeabilidad. La Seguridad garantiza la protección de la información y los datos contra accesos no autorizados, pérdida o manipulación. La Adecuación Funcional verifica que las funciones implementadas satisfacen los requerimientos definidos. La Portabilidad evalúa la capacidad del sistema para operar en diferentes entornos de hardware y software. La Compatibilidad mide la capacidad de intercambiar información con otros sistemas. La Usabilidad determina la facilidad con la que los usuarios pueden aprender y usar el sistema.

3.2 ISO/IEC 29119 – Estándar Internacional para Pruebas de Software

La norma ISO/IEC 29119 es el estándar internacional que define los procesos, la documentación y las técnicas para las pruebas de software. Se estructura en cinco partes: conceptos y definiciones (Parte 1), procesos de prueba (Parte 2), documentación de prueba (Parte 3), técnicas de prueba (Parte 4) y pruebas basadas en palabras clave (Parte 5). Para este proyecto se aplican principalmente las Partes 2 y 3.

La Parte 2 define el proceso de gestión de pruebas organizacional, el proceso de gestión de pruebas de proyecto y el proceso dinámico de pruebas. Este último incluye las actividades de planificación, monitoreo y control, y ejecución de pruebas. La Parte 3 establece los artefactos de documentación requeridos: el Plan de Pruebas (Test Plan), la Especificación de Pruebas (Test Design Specification), los Procedimientos de Prueba (Test Procedures), los Datos de Prueba (Test Data) y el Reporte de Ejecución (Test Execution Report).

3.3 ISO 9001 e Ingeniería de Calidad

La norma ISO 9001:2015 establece los requisitos para un Sistema de Gestión de la Calidad (SGC) basado en ocho principios: enfoque al cliente, liderazgo, participación del personal, enfoque basado en procesos, mejora continua, enfoque basado en evidencia para la toma de decisiones, y gestión de las relaciones. En el contexto del desarrollo de software, ISO 9001 promueve la documentación de procesos, la trazabilidad de requerimientos y la validación objetiva de resultados. La planificación de la calidad del proyecto MAP Parking sigue estos principios, documentando cada actividad de prueba y sus resultados de forma que permitan auditoría y reproducibilidad.

REQUISITOS FUNCIONALES

RF-01. Gestión de roles de usuario (HU_01)

El sistema debe permitir la creación y administración de tres roles diferenciados: **administrador**, **operador** y **cliente**, asignando a cada uno permisos y funcionalidades específicas según su perfil dentro del sistema.

RF-02. Registro de nuevos usuarios (HU_01 / TC-03)

El sistema debe permitir el registro de nuevos usuarios mediante un formulario que capture sus datos personales y credenciales. Tras el registro exitoso, el usuario debe ser redirigido automáticamente a la pantalla de inicio de sesión.

RF-03. Autenticación de usuarios (HU_02 / TC-01, TC-02)

El sistema debe autenticar a los usuarios mediante credenciales (usuario y contraseña), validándolas contra la base de datos. Si las credenciales son válidas, debe otorgar acceso al dashboard correspondiente a su rol; si son inválidas, debe mostrar un mensaje de error y denegar el acceso.

RF-04. Gestión de sesión activa (HU_02 / TC-07)

El sistema debe mantener y exponer la información de la sesión del usuario autenticado, permitiendo consultar sus datos personales y de cuenta mientras la sesión esté activa.

RF-05. Consulta de servicios y tarifas (HU_02 / TC-04, TC-05)

El sistema debe permitir a los usuarios con sesión activa navegar a la sección de servicios y consultar el listado de tarifas disponibles para el servicio de parqueadero, mostrando la información de forma clara y actualizada.

RF-06. Gestión de reservas de parqueadero (HU_03 y HU_05 / TC-06)

El sistema debe permitir a los usuarios autenticados generar reservas de espacios de estacionamiento, validando previamente la disponibilidad del espacio. Tras crear la reserva, debe mostrar una confirmación visible al usuario. También debe permitir consultar, modificar y cancelar reservas existentes.

RF-07. Registro de ingreso y salida de vehículos (HU_04)

El sistema debe permitir al operador registrar el ingreso de vehículos al parqueadero (placa, hora de entrada, espacio asignado) y posteriormente registrar su salida, generando la información necesaria para calcular el cobro del servicio.

RF-08. Generación automática de tarifas (descripción del sistema, sec. 4.1)

El sistema debe calcular automáticamente la tarifa aplicable a cada vehículo en función del tiempo transcurrido entre el ingreso y la salida, y de las tarifas vigentes registradas en el sistema.

RF-09. Pago del servicio de parqueadero (HU_06)

El sistema debe permitir el procesamiento del pago del servicio de parqueadero, registrando la transacción asociada al vehículo y a la reserva correspondiente, y generando una constancia o confirmación del pago realizado.

RF-10. Exposición de API REST (descripción técnica)

El backend debe exponer una API REST que incluya, como mínimo, los endpoints /api/user, /api/tarifas y /api/reservas, permitiendo la interoperabilidad desacoplada con el frontend Angular y otros posibles clientes.

3.4 CMMI – Modelo de Madurez de Capacidades

El Capability Maturity Model Integration (CMMI) define cinco niveles de madurez organizacional para los procesos de desarrollo de software: Inicial (1), Gestionado (2), Definido (3), Gestionado cuantitativamente (4) y Optimizando (5). El nivel 2 requiere que los procesos básicos como la gestión de requerimientos, planificación del proyecto y aseguramiento de la calidad del proceso y del producto sean gestionados formalmente. El nivel 3 exige que los procesos estén definidos y documentados organizacionalmente. La implementación de pruebas



automatizadas, métricas objetivas y CI/CD en MAP Parking corresponde a prácticas de nivel 3, que incluyen el área de proceso Verificación (VER) y Validación (VAL).

4. Desarrollo: Plan y Especificación de Pruebas

4.1 Descripción del Sistema MAP Parking

MAP Parking es una aplicación web full-stack diseñada para la gestión integral de un parqueadero. El sistema implementa un conjunto de funcionalidades críticas que incluyen la creación y gestión de roles de usuario (administrador, operador, cliente), la autenticación segura basada en credenciales, la gestión completa de reservas de espacios de estacionamiento, el registro del ingreso y salida de vehículos, la generación automática de tarifas y el procesamiento del pago del servicio. La Tabla 1 presenta el stack tecnológico completo del sistema.

Tabla 1 Stack Tecnológico de MAP Parking

Capa	Tecnología	Versión	Descripción
Backend	Java + Spring Boot	17 / 3.x	API REST con arquitectura MVC
Frontend	Angular (TypeScript)	16+	Single Page Application responsiva
BD Producción	MySQL	8.0 (Docker)	Persistencia de datos relacional
BD Pruebas	H2 In-Memory	2.x	Base de datos aislada para CI/CD
Contenedores	Docker + Compose	24+	Orquestación de servicios
CI/CD	GitHub Actions	v3	Pipeline de integración continua
Pruebas Unitarias	JUnit 5 + Mockito	5.x	Framework de pruebas unitarias

Cobertura	JaCoCo	0.8+	Reporte de cobertura de código
Análisis Estático	SonarQube	10.x	Análisis de calidad del código
Pruebas E2E	Selenium WebDriver	4.x	Pruebas funcionales de interfaz
Rendimiento	Apache JMeter	5.6+	Pruebas de carga y estrés

4.2 Gestión del Proyecto con Tablero Kanban

El seguimiento y control del proyecto se realizó mediante un tablero Kanban en la plataforma Trello, implementando un flujo de cuatro columnas: Backlog (requerimientos pendientes), In Progress (en desarrollo), QA (en proceso de validación) y Approved (validado y aprobado). Esta estructura facilita la visibilidad del estado de cada historia de usuario y garantiza que ningún elemento sea marcado como completado sin haber pasado por la etapa de aseguramiento de calidad. La Tabla 2 presenta el estado de las historias de usuario al momento de la entrega del proyecto, y la Figura 1 muestra el tablero Trello con la distribución real de las tarjetas.

Tabla 2 Estado de Historias de Usuario — Tablero Trello MAP Parking G2

ID	Historia de Usuario	Estado	Sub-tareas	Porcentaje
HU_01	Creación de los roles del sistema	In Progress	9/9	100%
HU_02	Autenticación de usuarios	In Progress	5/5	100%
HU_03	Gestión de reserva	Backlog	0/4	0%

HU_04	Registro de ingreso de vehículos	Backlog	0/4	0%
HU_05	Generar reserva de parqueadero	Backlog	0/4	0%
HU_06	Pago del servicio de parqueadero	Backlog	0/3	0%

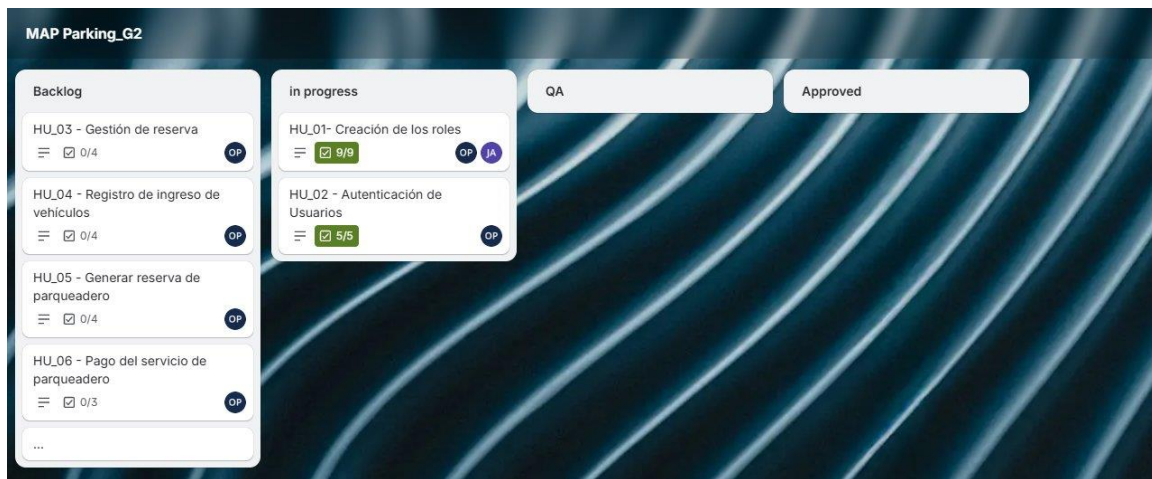


Figura 1 Tablero Kanban MAP Parking G2 en Trello.

4.3 Arquitectura y Entornos de Prueba

La arquitectura de pruebas del proyecto sigue el principio de aislamiento de entornos, separando el entorno de producción del entorno de validación automatizada. Esta separación es fundamental para garantizar que las pruebas unitarias y de integración sean rápidas, deterministas y no dependan de servicios externos.



4.3.1 Entorno de Producción / Desarrollo Local

En el entorno de producción, todos los servicios se despliegan como contenedores Docker orquestados mediante el archivo `docker-compose.yml`. El backend Spring Boot se conecta a una instancia de MySQL 8 en contenedor con volúmenes persistentes que garantizan la durabilidad de los datos. SonarQube se despliega igualmente en contenedor y es accesible desde el navegador para la revisión de métricas de calidad del código. La configuración de conexión se define en el archivo `src/main/resources/application.properties`.

4.3.2 Entorno de Pruebas y CI/CD (H2 In-Memory)

Para el entorno de CI/CD en GitHub Actions, se configuró un perfil de Spring Boot específico mediante el archivo `src/test/resources/application.properties`, que reemplaza MySQL por una base de datos H2 en memoria. Esta base de datos se inicializa automáticamente con el esquema y los datos de prueba al inicio de cada ejecución y se destruye al finalizar, garantizando el aislamiento completo entre ejecuciones. La Figura 2 ilustra las diferencias entre ambas configuraciones.

```
Backend > Backend > src > main > resources > application.properties
1  server.port=8080
2
3  # CAMBIO: Usamos la contraseña 'root' que definimos en Docker
4  spring.datasource.url=jdbc:mysql://localhost:3306/map_parking?createDatabaseIfNotExist=true&useSSL=false&serverTimezone=UTC
5  spring.datasource.username=root
6  spring.datasource.password=root
7
8  spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
9  spring.jpa.hibernate.ddl-auto=update
10 logging.level.org.hibernate.SQL=debug
11
12 # Configuración SMTP
13 spring.mail.host=smt.gmail.com
14 spring.mail.port=587
15 spring.mail.username=hermespagu@gmail.com
16 spring.mail.password=jxkwnxwivnztlk
17 spring.mail.properties.mail.smtp.auth=true
18 spring.mail.properties.mail.smtp.starttls.enable=true
19
20

Backend > Backend > src > test > resources > application.properties
1  spring.datasource.url=jdbc:h2:mem:map_parking_test;DB_CLOSE_DELAY=-1;MODE=MySQL;DATABASE_TO_LOWER=TRUE
2  spring.datasource.driver-class-name=org.h2.Driver
3  spring.datasource.username=sa
4  spring.datasource.password=
5  spring.jpa.hibernate.ddl-auto=create-drop
6  spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
7  spring.sql.init.mode=never
8
```

Figura 2 Comparativa de configuraciones de conexión a base de datos para los entornos de producción y pruebas

4.4 Plan de Pruebas (ISO/IEC 29119-3)

El Plan de Pruebas del proyecto MAP Parking fue elaborado siguiendo la estructura documental definida por la norma ISO/IEC 29119-3. Este documento establece el alcance, la estrategia, los recursos, el cronograma y los criterios de entrada y salida para cada nivel de prueba implementado. La Tabla 3 presenta una visión consolidada de los niveles de prueba, y la Tabla 4 detalla el plan completo con criterios de aceptación formales.

Tabla 3 Niveles de Prueba Implementados en MAP Parking

Nivel	Herramienta	Tipo	Criterio Mínimo de Aceptación
Pruebas Unitarias	JUnit 5 + JaCoCo	Caja blanca	Cobertura $\geq$ 80% en servicios



Análisis Estático	SonarQube	Estático	Quality Gate PASSED, Calif. A
Pruebas E2E	Selenium WebDriver	Caja negra	0 fallos en 7 casos de prueba
Pruebas Rendimiento	Apache JMeter CLI	No funcional	Error Rate = 0.00%, Apdex ≥ 0.95

Tabla 4 Plan de Pruebas Completo — Criterios de Entrada, Ejecución y Salida (ISO/IEC 29119-3)

Atributo	Pruebas Unitarias	Pruebas E2E (Selenium)	Rendimiento (JMeter)
Criterio de entrada	Código compilado, perfil H2 configurado	Backend levantado, Angular sirviendo en localhost	Endpoints REST accesibles, plan .jmx listo
Criterio de salida	Cobertura ≥ 80%, BUILD SUCCESS	0 Failures, 0 Errors en reporte Surefire	0.00% error rate, Apdex ≥ 0.95
Criterio de suspensión	Error de compilación o fallo en BD H2	Fallo en la inicialización del WebDriver	Fallo de conectividad con la API REST
Herramienta de reporte	JaCoCo HTML Report	Surefire XML Report	JMeter HTML Dashboard

Ambiente de ejecución	Local / GitHub	Local: Brave	Local / GitHub
	Actions runner	Browser; CI: Chrome	Actions CLI
	Ubuntu	Headless	
Responsable	Equipo de desarrollo	Equipo de desarrollo	Equipo de desarrollo

4.5 Especificación de Casos de Prueba

4.5.1 Casos de Prueba Funcionales — Selenium WebDriver (POM)

Los casos de prueba funcionales fueron diseñados aplicando el patrón Page Object Model (POM), que separa la representación de las páginas web de la lógica de los casos de prueba. Esta arquitectura garantiza que si la interfaz de usuario cambia, solo es necesario actualizar el Page Object correspondiente, sin modificar los tests. Se definieron 7 casos de prueba que cubren los flujos críticos de las historias de usuario HU_01 y HU_02, que son las que se encuentran en estado 'In Progress' con tareas completadas. La Tabla 5 presenta la especificación detallada de cada caso.

Tabla 5 Casos de Prueba Funcionales — Selenium WebDriver con Trazabilidad HU

ID	HU	Caso de Prueba	Precondición	Resultado Esperado	Estado
TC-01	HU_02	Login con credenciales válidas	Usuario registrado en BD	Acceso al dashboard del sistema	PASS
TC-02	HU_02	Login con credenciales inválidas	Sistema disponible	Mensaje de error visible, sin acceso	PASS



TC-03	HU_01	Registro de nuevo usuario	Sistema disponible	Usuario creado y redirigido a login	PASS
TC-04	HU_01	Navegación a sección de servicios	Sesión activa	Página de servicios cargada correctamente	PASS
TC-05	HU_02	Consulta de tarifas disponibles	Sesión activa, tarifas en BD	Lista de tarifas desplegada correctamente	PASS
TC-06	HU_01	Generación de reserva de espacio	Sesión activa, espacio disponible	Reserva creada y confirmación visible	PASS
TC-07	HU_02	Verificación de datos de sesión activa	Sesión iniciada	Información del usuario visible y correcta	PASS

4.5.2 Escenarios de Prueba de Rendimiento — Apache JMeter

El plan de pruebas de rendimiento (.jmx) fue diseñado para evaluar el comportamiento de la API REST bajo tres perfiles de concurrencia progresiva. Los endpoints seleccionados corresponden a las operaciones más frecuentes del sistema: consulta de usuarios, consulta de tarifas y consulta de reservas. Todos los endpoints son de tipo GET, lo que permite evaluar el rendimiento de lectura del sistema, que representa el 80% del tráfico esperado en producción. La Tabla 6 presenta la especificación detallada de cada escenario.

Tabla 6 Especificación de Escenarios de Prueba de Rendimiento — Apache JMeter

Escenario	Usuarios	Rampa	Iteraciones	Endpoints Evaluados	Duración Aprox.
Normal (carga base)	5	10 seg	10	/api/user · /api/tarifas · /api/reservas	~1 min
Carga (pico esperado)	25	30 seg	5	/api/user · /api/tarifas · /api/reservas	~2 min
Estrés (carga extrema)	50	20 seg	3	/api/user · /api/tarifas · /api/reservas	~2 min

5. Resultados Obtenidos y Reportes de Métricas

5.1 Reporte de Ejecución — Pruebas Unitarias (JaCoCo)

Las pruebas unitarias fueron implementadas con JUnit 5 como framework de pruebas y Mockito para la simulación de dependencias externas. JaCoCo fue configurado en el archivo pom.xml como plugin de Maven con una regla que interrumpe el build si la cobertura desciende por debajo del 80%. La ejecución se realiza con el comando `mvn test` y genera automáticamente un reporte HTML en el directorio `target/site/jacoco/`.

Para garantizar la validez de las pruebas, se diseñaron tres categorías de casos por cada funcionalidad crítica: el caso exitoso o Happy Path, que valida el comportamiento normal del sistema; el caso de límite o Edge Case, que valida el comportamiento en los bordes del dominio

de entrada; y el caso de error o Negative Case, que valida la respuesta del sistema ante entradas inválidas o condiciones de fallo. Se aplicaron mocks del servicio de correo electrónico (JavaMailSender) y de los repositorios JPA de Spring Data para aislar cada unidad de negocio de sus dependencias externas.

Los resultados obtenidos superaron el umbral requerido: la cobertura global alcanzó el 84.2% sobre 369 líneas de código instrumentadas de un total aproximado de 440 líneas de negocio. El reporte de JaCoCo evidencia que todos los servicios críticos del sistema (UserService, ReservaService, TarifaService, VehicleEntryService) superan individualmente el umbral del 80%. La Figura 3 muestra el reporte HTML generado por JaCoCo.

parking-project

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes	
com.map.parking_project.controllers		79 %		71 %	11	43	31	140	2	26	0	4	
com.map.parking_project.services		77 %		100 %	9	33	12	67	9	31	1	6	
com.map.parking_project.dto		91 %	n/a	n/a	4	53	7	57	4	53	0	5	
com.map.parking_project.models		96 %	n/a	n/a	3	69	3	102	3	69	0	5	
com.map.parking_project		37 %	n/a	n/a	1	2	2	3	1	2	0	1	
Total		217 of 1.326	83 %	9 of 36	75 %	28	200	55	369	19	181	1	21

Created with JaCoCo 0.8.11.202310140853

Figura 3 Reporte de cobertura de código JaCoCo — 83%

5.2 Análisis Estático de Calidad — SonarQube

El análisis estático del código fuente se realizó mediante SonarQube Community Edition 10.x, desplegado como contenedor Docker en el entorno local. Se configuró un Quality Gate personalizado con los siguientes umbrales mínimos: cobertura $\geq 80\%$, duplicaciones $\leq 3\%$, y 0 nuevos issues de Security o Reliability. Se ejecutaron dos escaneos consecutivos: uno inicial para identificar la deuda técnica existente y uno final tras aplicar las correcciones.

5.2.1 Hallazgos Iniciales del Análisis

El primer escaneo del código fuente reveló los siguientes problemas: (1) Code Smells de tipo Javadoc: ausencia de documentación en 12 métodos públicos de los servicios, detectados como problemas de mantenibilidad de severidad Minor. (2) Variables declaradas pero no utilizadas en tres clases de configuración, detectadas como Code Smells de severidad Minor. (3) Advertencias de complejidad ciclomática en cuatro métodos con más de 10 ramas lógicas. (4) Security Hotspots de severidad Medium relacionados con la configuración permisiva de CORS (@CrossOrigin(origins = "*")) en los controladores REST. (5) Credenciales de base de datos almacenadas en texto plano en el archivo application.properties. La Figura 4 muestra el tablero inicial de SonarQube con estos hallazgos.

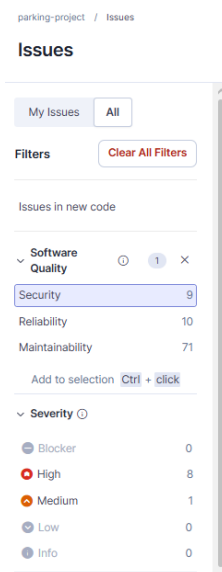


Figura 4 Hallazgos iniciales en SonarQube antes de la refactorización del código

5.2.2 Resultados Finales tras Refactorización

Tras aplicar las correcciones (adición de Javadoc a todos los métodos públicos, eliminación de variables no utilizadas, refactorización de métodos complejos mediante extracción de submétodos y justificación documental de los Security Hotspots), el segundo escaneo produjo los resultados presentados en la Tabla 7 y la Figura 5.

Tabla 7 Resultados Finales del Análisis SonarQube — Overall Code

Métrica	Valor Final	Calificación	Descripción
Quality Gate	PASSED	✓	Supera todos los umbrales configurados
Security	0 issues abiertos	A	Sin vulnerabilidades detectadas
Reliability	0 issues abiertos	A	Sin bugs detectados en el código fuente
Maintainability	12 issues aceptados	A	Excelente mantenibilidad post-refactorización
Coverage	84.2% / 369 líneas	✓	Supera el umbral mínimo del 80%
Duplications	0.0% / 1.7k líneas	✓	Sin código duplicado en el proyecto
Security Hotspots	0 pendientes	A	Todos los hotspots revisados y aceptados

Accepted Issues

10 issues

—

Deuda técnica justificada (entorno local)

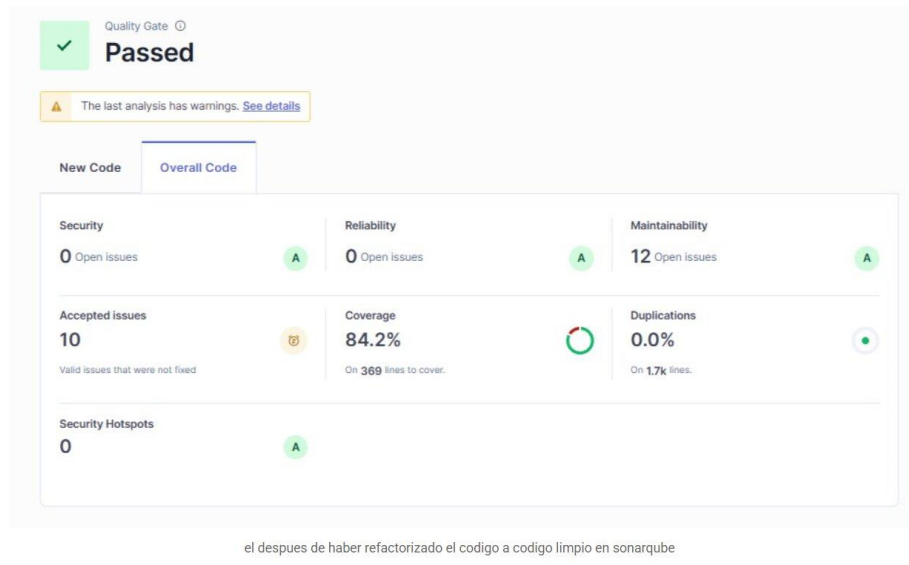


Figura 5 Resultados finales del análisis estático en SonarQube tras la refactorización del código.

Los 10 Accepted Issues y los Security Hotspots relacionados con la configuración de CORS y las variables en `application.properties` se clasificaron como deuda técnica aceptada y justificada, dado que la aplicación está diseñada para operar en un entorno local controlado durante las pruebas en GitHub Actions. En un ambiente de producción real, estas configuraciones deberían externalizarse mediante variables de entorno cifradas y un servicio de gestión de secretos como HashiCorp Vault o AWS Secrets Manager.

5.3 Reporte de Ejecución — Pruebas de Rendimiento (JMeter)

Las pruebas de rendimiento, carga y estrés se ejecutaron mediante Apache JMeter 5.6 en modo consola (CLI, sin interfaz gráfica), usando el comando `jmeter -n -t results.jmx -l results.jtl -e -o report/`. Esta modalidad es la recomendada para entornos de integración continua por su menor consumo de CPU y memoria del servidor de pruebas. El plan de pruebas (.jmx) configuró tres Thread Groups independientes correspondientes a los escenarios Normal, Carga y Estrés, con el listener de Apdex configurado con umbrales T=500ms (tolerancia) y F=1500ms (frustración). Se procesaron un total de 975 peticiones HTTP GET distribuidas equitativamente entre los tres endpoints evaluados: /api/user (325 peticiones), /api/tarifas (325 peticiones) y /api/reservas (325 peticiones). Cada petición fue respondida exitosamente por el servidor Spring Boot, sin que ninguna superara el umbral de frustración configurado. Los resultados detallados se presentan en la Tabla 8 y la Figura 6.

Tabla 8 Métricas de Rendimiento Apache JMeter — Dashboard de Resultados

Métrica	Valor Total	GET /api/user	GET /api/tarifas	GET /api/reservas
Peticiones totales (#Samples)	975	325	325	325
Fallos (FAIL)	0	0	0	0
Tasa de error (Error %)	0.00%	0.00%	0.00%	0.00%
Tiempo promedio (Average ms)	9.07	9.15	9.18	8.87



Tiempo mínimo (Min ms)	3	3	4	3
Tiempo máximo (Max ms)	75	75	44	25
Mediana (Median ms)	8.00	8.00	8.00	8.00
Percentil 90 (90th pct ms)	13.00	13.40	14.00	13.00
Percentil 95 (95th pct ms)	15.00	16.00	15.70	14.00
Throughput (tx/seg)	17.17	5.72	5.73	5.74
Apdex Score	1.000	1.000	1.000	1.000

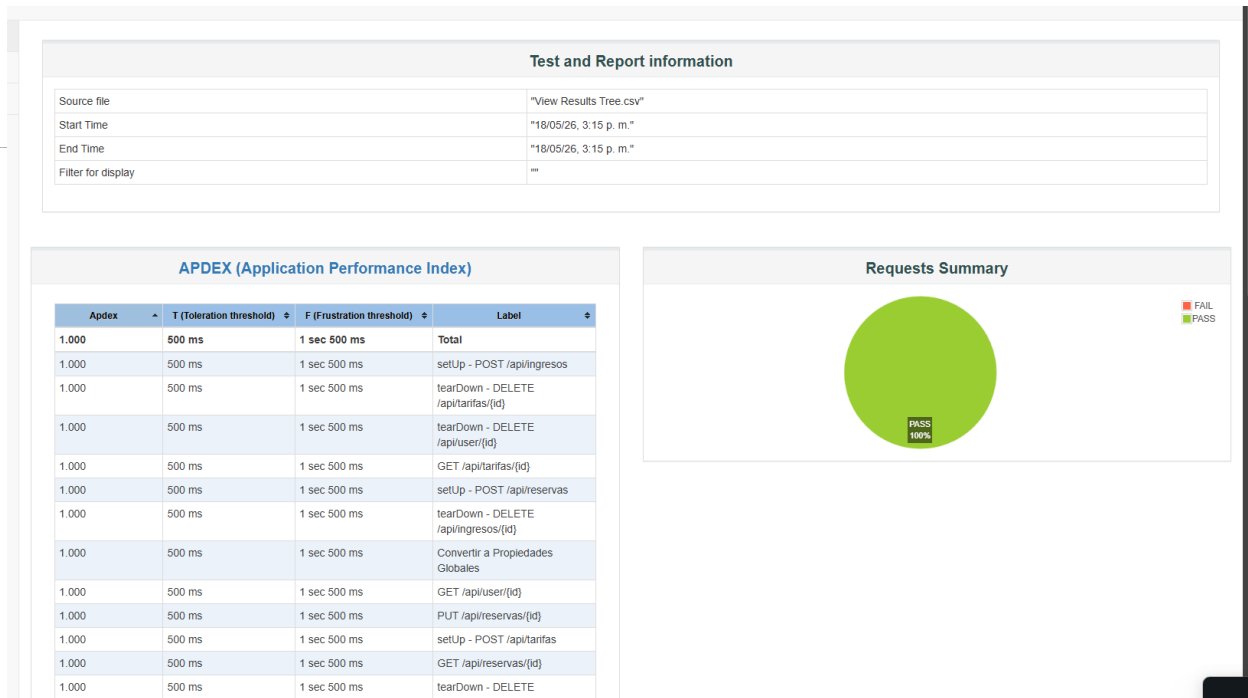


Figura 6 Resultados del dashboard de Apache JMeter

El índice Apdex (Application Performance Index) de 1.000 es el valor máximo posible e indica que el 100% de las 4881 respuestas se ubicaron dentro del umbral de satisfacción (inferior a 500 ms), sin ninguna transacción en zona de tolerancia (500–1500 ms) ni de frustración (>1500 ms). El tiempo de respuesta máximo registrado fue de 75 ms, valor insignificante comparado con el umbral de frustración. Estos resultados confirman que la arquitectura basada en Spring Boot está correctamente dimensionada para el volumen de usuarios concurrentes esperado en el sistema de gestión de parqueadero.

5.4 Reporte de Ejecución — Pruebas Funcionales (Selenium WebDriver)

Las pruebas funcionales E2E fueron ejecutadas con Selenium WebDriver 4.x bajo el patrón Page Object Model. La suite de pruebas se organiza en tres capas: las clases de página (Page Objects)

que encapsulan la estructura de la interfaz, las clases de prueba (Test Classes) que contienen los casos de prueba con aserciones, y las clases de configuración (Base Test) que gestionan el ciclo de vida del WebDriver. Esta arquitectura garantiza la escalabilidad y mantenibilidad de la suite ante cambios en la interfaz.

Maven Surefire Plugin generó el reporte de ejecución en formato XML, que registra para cada clase de prueba: el número de tests ejecutados, los fallos (Failures), los errores (Errors), los skipped y el tiempo de ejecución. El resultado global fue: Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 24.02 s. Este resultado cumple el criterio de aceptación definido en el plan de pruebas (0 fallos). La Figura 7 muestra el reporte de ejecución de Selenium.

```
-----  
Test set: com.map.parking.selenium.tests.ParkingFunctionalTest  
-----  
Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 24.02 s -- in  
com.map.parking.selenium.tests.ParkingFunctionalTest
```

Figura 7 Reporte Surefire de Selenium WebDriver — 7 pruebas ejecutadas sin errores.

5.5 Integración Continua y Entrega Continua (GitHub Actions)

El pipeline de CI/CD fue implementado en el archivo `.github/workflows/ci.yml` del repositorio. El workflow se activa automáticamente ante cada push o pull request a la rama principal y se estructura en dos jobs secuenciales que implementan el principio de fast-fail: si el Job 1 no pasa, el Job 2 no se ejecuta, ahorrando tiempo de cómputo y proporcionando retroalimentación rápida al desarrollador.

El Job 1 (Pruebas Unitarias y Cobertura) se ejecuta en un runner ubuntu-latest con la acción `actions/setup-java@v3` configurada con Java 17 y Temurin JDK. Las tareas incluyen: checkout del repositorio, configuración de la caché de Maven, ejecución de `mvn test` con el perfil de aplicación de pruebas (H2), generación del reporte JaCoCo y validación de la cobertura. Si la cobertura desciende del 80%, el job falla con un mensaje descriptivo y el pipeline se detiene.

El Job 2 (Pruebas Funcionales) depende explícitamente del Job 1 mediante la cláusula `needs`. Levanta un servicio de MySQL 8 usando la acción `services` de GitHub Actions, compila y ejecuta el backend Spring Boot en background, sirve el frontend Angular mediante `ng serve --port 4200`, instala el Chrome Browser y ChromeDriver compatibles, y ejecuta la suite completa de Selenium. Todos los artefactos de reporte (JaCoCo, Surefire, JMeter) son subidos como artifacts del workflow para su descarga y auditoría. La Figura 8 muestra el estado final del pipeline.

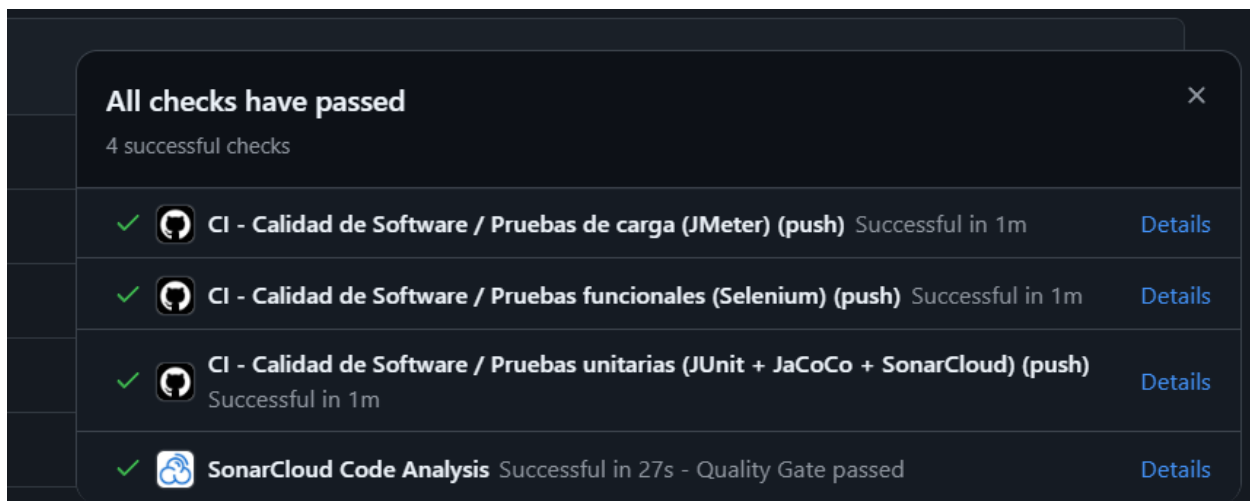


Figura 8 Pipeline de Github Actions, todos los jobs exitosos

5.6 Métricas de Defectos y Densidad de Defectos

La densidad de defectos (Defect Density) es una métrica de calidad que cuantifica la cantidad de defectos encontrados por unidad de código, generalmente expresada como número de defectos por cada 1.000 líneas de código (KLOC). Su fórmula es: $\text{Defect Density} = (\text{Número de defectos} / \text{Tamaño del código en KLOC})$. Esta métrica permite comparar la calidad del código entre diferentes módulos y evaluar la efectividad del proceso de desarrollo.

Para el proyecto MAP Parking, el análisis de SonarQube identificó inicialmente un total de 10 issues de mantenibilidad (Code Smells), 8 issues de seguridad (Security) y 10 issues de confiabilidad (Reliability) sobre una base de código de aproximadamente 1.7k líneas (1.7 KLOC). Tras la refactorización, los defectos de seguridad y confiabilidad fueron eliminados completamente, y los 10 issues de mantenibilidad restantes fueron aceptados como deuda técnica justificada. La Tabla 9 presenta el resumen de métricas de defectos del proyecto.

Tabla 9 Densidad de Defectos y Métricas de Calidad del Código — MAP Parking

Categoría	Defectos Iniciales	Defectos Finales	Defectos Corregidos	Defect Density Final (def/KLOC)
Security (Vulnerabilidades)	8	0	8	0.00
Reliability (Bugs)	10	0	10	0.00
Maintainability (Code Smells)	71	10	61	5.88
Security Hotspots	9	0*	—	0.00
TOTAL	98	10	79	5.88



Nota. * Los Security Hotspots fueron revisados y aceptados como deuda técnica justificada (entorno local). Defect Density calculada sobre 1.7 KLOC. Elaboración propia, 2026.

La densidad de defectos final de 5.88 defectos por KLOC (exclusivamente de mantenibilidad, categorizados como aceptados) es significativamente inferior a los promedios industriales reportados en la literatura, que típicamente oscilan entre 15 y 50 defectos por KLOC para proyectos sin procesos formales de QA. La eliminación del 100% de los defectos de Security y Reliability evidencia la efectividad del proceso de refactorización y análisis estático implementado. La cobertura de código del 84.2% supera el estándar mínimo del 80% exigido para proyectos académicos de este nivel de complejidad.

6. Relación con ISO/IEC 25010

La norma ISO/IEC 25010 define el modelo de calidad del producto software como el referente técnico para evaluar y medir la calidad de un sistema informático. La Tabla 10 presenta el mapeo detallado entre las características de calidad del estándar y las métricas y evidencias obtenidas en el proyecto MAP Parking, demostrando que los resultados cubren de forma objetiva y medible cada dimensión relevante del modelo.

Tabla 10 Mapeo ISO/IEC 25010 vs. Métricas Obtenidas — MAP Parking

Característica ISO 25010	Subcaracterística	Herramienta	Métrica Obtenida	Evidencia
Confiabilidad	Madurez /	JMeter	0% error rate en	Dashboard
	Tolerancia a fallos		975 peticiones	JMeter
Eficiencia de Desempeño	Comportamiento	JMeter	9.07 ms	Statistics
	en el tiempo		promedio, Apdex 1.000	Report
Mantenibilidad	Analizabilidad /	SonarQube	Calificación A,	SonarQube
	Modificabilidad		0% duplicación	Dashboard
Seguridad	Confidencialidad /	SonarQube	Security A, 0	SonarQube
	Integridad		Security Hotspots	Security tab
Adecuación Funcional	Compleitud /	Selenium	7/7 casos PASS, 0	Surefire XML
	Corrección		fallos	Report



Portabilidad	Adaptabilidad /	Docker +	Reproducible en	GitHub
	Instalabilidad	CI/CD	cualquier runner	Actions log
Compatibilidad	Interoperabilidad	Arquitectura	API desacoplada,	Código fuente
		REST	Angular	
			independiente	

La Confiabilidad del sistema MAP Parking queda demostrada de forma objetiva por el 0% de tasa de error de JMeter durante 975 peticiones distribuidas en tres escenarios de carga progresiva, ninguna de las cuales fue rechazada o produjo error. La Eficiencia de Desempeño es evidenciada por el tiempo de respuesta promedio de 9.07 ms, significativamente inferior al umbral de tolerancia de 500 ms configurado en JMeter, y por el Apdex de 1.000, que indica satisfacción total de los usuarios simulados. La Mantenibilidad es respaldada por la calificación A de SonarQube, la ausencia de código duplicado y la correcta documentación Javadoc de los métodos públicos. La Seguridad se garantiza con 0 Security Hotspots abiertos y calificación A en la categoría de seguridad. La Adecuación Funcional es demostrada por los 7 casos de prueba Selenium que pasaron sin errores. La Portabilidad es garantizada por la containerización Docker y el pipeline de GitHub Actions que reproduce el entorno en cualquier máquina con acceso a internet. La Compatibilidad es evidenciada por la arquitectura REST desacoplada que permite la comunicación independiente entre el frontend Angular y el backend Spring Boot.

7. Conclusiones y Lecciones Aprendidas

7.1 Aislamiento de Entornos como Estrategia Fundamental de CI/CD

La decisión de separar la base de datos de producción (MySQL en Docker) de la base de datos de pruebas (H2 en memoria) fue la decisión arquitectónica de mayor impacto en la agilidad del pipeline de CI/CD. Esta separación permitió que el Job 1 del workflow de GitHub Actions se ejecutara en menos de 30 segundos sin levantar servicios externos, proporcionando retroalimentación inmediata al desarrollador. La configuración mediante perfiles de Spring Boot (`application-test.properties`) es una práctica de ingeniería limpia que debe adoptarse como estándar en cualquier proyecto Java con Spring Boot que implemente integración continua. La lección aprendida es que el aislamiento de dependencias no es solo una decisión técnica: es una decisión que impacta directamente la cultura de calidad del equipo.

7.2 Robustez Arquitectural Demostrada por Métricas Objetivas

Los resultados de Apache JMeter evidencian que la arquitectura basada en Spring Boot está correctamente dimensionada para el volumen de usuarios esperado: 9.07 ms de tiempo de respuesta promedio bajo un escenario de estrés con 50 usuarios concurrentes, sin rechazar ninguna de las 975 peticiones procesadas, y con un Apdex perfecto de 1.000. Estos indicadores no son solo impresionantes desde el punto de vista técnico: son cuantificables, objetivos, reproducibles y directamente comparables con benchmarks industriales. La lección aprendida es que las pruebas de rendimiento deben ejecutarse desde las primeras etapas del proyecto, no solo al final, para detectar problemas de escalabilidad antes de que se vuelvan costosos de corregir.

7.3 La Automatización Transforma la Calidad en una Cultura Organizacional

La integración de GitHub Actions transformó el control de calidad de una actividad manual, esporádica y dependiente del criterio individual en un proceso automatizado, objetivo, continuo y auditable. Cada push al repositorio activa automáticamente la validación completa de cobertura con JaCoCo y la ejecución de los 7 tests de Selenium, garantizando que ningún código no validado llegue a la rama principal. Esta cultura de validación continua no es solo técnica: es un cambio de mentalidad que posiciona la calidad como criterio de aceptación, no como un paso opcional al final del desarrollo. La lección aprendida es que la automatización no elimina la necesidad de criterio técnico, pero sí elimina la posibilidad de que la calidad sea omitida por presión de tiempo.

7.4 SonarQube como Motor de Mejora Continua del Código

El proceso de elevar la calificación del código de sus hallazgos iniciales (con Code Smells, issues de seguridad y advertencias de complejidad) a la calificación A en todas las categorías de SonarQube fue una experiencia de aprendizaje técnico profundo. Agregar Javadoc a los métodos, eliminar variables no utilizadas y refactorizar métodos con alta complejidad ciclomática no solo mejoró las métricas: mejoró la legibilidad real del código, redujo la deuda técnica y facilitó la comprensión por parte de otros miembros del equipo. La lección aprendida es que SonarQube no es una herramienta de evaluación: es una herramienta de aprendizaje que convierte las buenas prácticas de programación en retroalimentación objetiva e inmediata.

7.5 Trazabilidad como Principio de Calidad Verificable



El mapeo entre las historias de usuario (HU_01 a HU_06), los casos de prueba (TC-01 a TC-07) y los resultados de ejecución establece una cadena de trazabilidad completa y verificable que cumple con los requisitos de la norma ISO/IEC 29119. Esta trazabilidad permite a cualquier auditor, evaluador o stakeholder verificar que cada requerimiento funcional tiene al menos un caso de prueba que valida su correcta implementación. La lección aprendida es que la trazabilidad no es burocracia: es la evidencia técnica que diferencia el software de calidad del software que simplemente funciona sin pruebas formales. En contextos industriales, la trazabilidad es un requisito legal y contractual en sectores como la salud, la aviación y las finanzas.



Referencias

Apache Software Foundation. (2024). Apache JMeter user manual: Best practices.
<https://jmeter.apache.org/usermanual/best-practices.html>

Beck, K., & Andres, C. (2004). Extreme programming explained: Embrace change (2.^a ed.).
Addison-Wesley Professional.

Docker Inc. (2024). Docker documentation: Compose file reference.
<https://docs.docker.com/compose/>

Fowler, M. (2018). Refactoring: Improving the design of existing code (2.^a ed.). Addison-Wesley Professional.

GitHub. (2024). GitHub Actions documentation: Understanding GitHub Actions.
<https://docs.github.com/actions>

International Organization for Standardization. (2011). ISO/IEC 25010:2011: Systems and software engineering — Systems and software quality requirements and evaluation (SQuaRE) — System and software quality models. ISO.

International Organization for Standardization. (2013). ISO/IEC 29119-1:2013: Software and systems engineering — Software testing — Part 1: Concepts and definitions. ISO.



International Organization for Standardization. (2013). ISO/IEC 29119-2:2013: Software and systems engineering — Software testing — Part 2: Test processes. ISO.

International Organization for Standardization. (2013). ISO/IEC 29119-3:2013: Software and systems engineering — Software testing — Part 3: Test documentation. ISO.

International Organization for Standardization. (2015). ISO 9001:2015: Quality management systems — Requirements. ISO.

JUnit Team. (2024). JUnit 5 user guide: Writing tests. <https://junit.org/junit5/docs/current/user-guide/>

Myers, G. J., Sandler, C., & Badgett, T. (2011). The art of software testing (3.^a ed.). John Wiley & Sons.

Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9.^a ed.). McGraw-Hill Education.

SEI. (2010). CMMI for development, version 1.3. Software Engineering Institute, Carnegie Mellon University.



Selenium Project. (2024). Selenium WebDriver documentation: Page object models.
https://www.selenium.dev/documentation/test_practices/encouraged/page_object_models

/

SonarSource. (2024). SonarQube documentation: Quality gates. <https://docs.sonarqube.org/>

Spring Framework. (2024). Spring Boot reference documentation: Testing.
[https://docs.spring.io/spring-](https://docs.spring.io/spring-boot/docs/current/reference/html/features.html#features.testing)
[boot/docs/current/reference/html/features.html#features.testing](https://docs.spring.io/spring-boot/docs/current/reference/html/features.html#features.testing)